

Quintic Doc

Summary: At each control cycle/step, the desired pose at a look-ahead time is computed to obtain error twist which is then mapped to joint velocities using differential inverse kinematics.

Quintic Polynomial for Position

Let $p(t) \in \mathbb{R}$ denote end-effector position. A quintic trajectory has the form

$$p(t) = a_0 + a_1t + a_2t^2 + a_3t^3 + a_4t^4 + a_5t^5.$$

(Quintic polynomial was chosen based on a prior team meeting where we discussed it will be good to have smooth acceleration profiles (S-shapeded).)

To guarantee continuous position, velocity, and acceleration, the boundary conditions

$$p(0) = p_0, \quad p(T) = p_T, \quad \dot{p}(0) = v_0, \quad \dot{p}(T) = v_T, \quad \ddot{p}(0) = a_0, \quad \ddot{p}(T) = a_T$$

yield the linear system $A\mathbf{a} = \mathbf{b}$, from which the coefficients $\mathbf{a} = [a_0, \dots, a_5]^\top$ are solved. The translational trajectory in $\mathbb{R}^3$ is obtained by fitting three independent quintics to the x, y, z coordinates of the horizon points. (There should be a way to vectorize this instead of doing one axis at a time like the current code does.)

The instantaneous desired position and linear velocity follow directly:

$$p(t) = \sum_{i=0}^5 a_i t^i, \quad \dot{p}(t) = \sum_{i=1}^5 i a_i t^{i-1}.$$

SLERP for Orientation

At the same timestep *at_time_t* that position is computed, orientation is obtained via spherical linear interpolation (SLERP). This is pretty standard and there's not much to add in here. I used scipy's SLERP function.

Pose Error

The desired pose at timestep *at_time_t* is constructed from the position and orientation computed above. Error twist is obtained by comparing the desired pose to the current end-effector pose.

Staying consistent with *cartesianDualArmPathFollowingControlLoop()*, I included the multiplication by 2 for the orientation. I actually wanted to bring up why this was needed, my best guess is empirical tests revealed this scaling to be needed.

Differential Inverse Kinematics

This is exactly the same as in *cartesianDualArmPathFollowingControlLoop()*. I also ensured the same return type so that my implementation can be a drop-in replacement.